

MODUL PRAKTIKUM

Mata Kuliah

# DevSecOps

Static Application Security Testing  
(SAST) dengan SonarQube  
— Project PHP/Laravel —

Jago IT Berkarakter



**SEKOLAH TINGGI TEKNOLOGI  
TERPADU NURUL FIKRI**



# Informasi Modul Praktikum

 **Mata Kuliah**

DevSecOps — S1 Teknik Informatika

 **Topik**

Static Application Security Testing (SAST) dengan SonarQube

 **Durasi**

150 menit (2 sks praktikum)

 **Mode**

Kerja kelompok (3-4 mahasiswa per kelompok)

 **Capaian**

Mahasiswa mampu menginstal, menjalankan scan, dan menginterpretasi laporan SAST

# Tujuan Praktikum & Tools yang Dibutuhkan

## Tujuan Praktikum

- 1 Menginstal SonarQube menggunakan Docker Compose
- 2 Membuat dan mengkonfigurasi project PHP/Laravel untuk di-scan SAST
- 3 Menjalankan SonarScanner dan membaca hasil laporan
- 4 Mengidentifikasi & mendokumentasikan temuan keamanan
- 5 Memahami severity level dan prioritas remediasi

## Tools yang Dibutuhkan

|                |      |
|----------------|------|
| Docker Desktop | v24+ |
|----------------|------|

|                |      |
|----------------|------|
| Docker Compose | v2.x |
|----------------|------|

|           |                   |
|-----------|-------------------|
| SonarQube | Community Edition |
|-----------|-------------------|

|                  |      |
|------------------|------|
| SonarScanner CLI | v5.x |
|------------------|------|

|     |      |
|-----|------|
| Git | v2.x |
|-----|------|

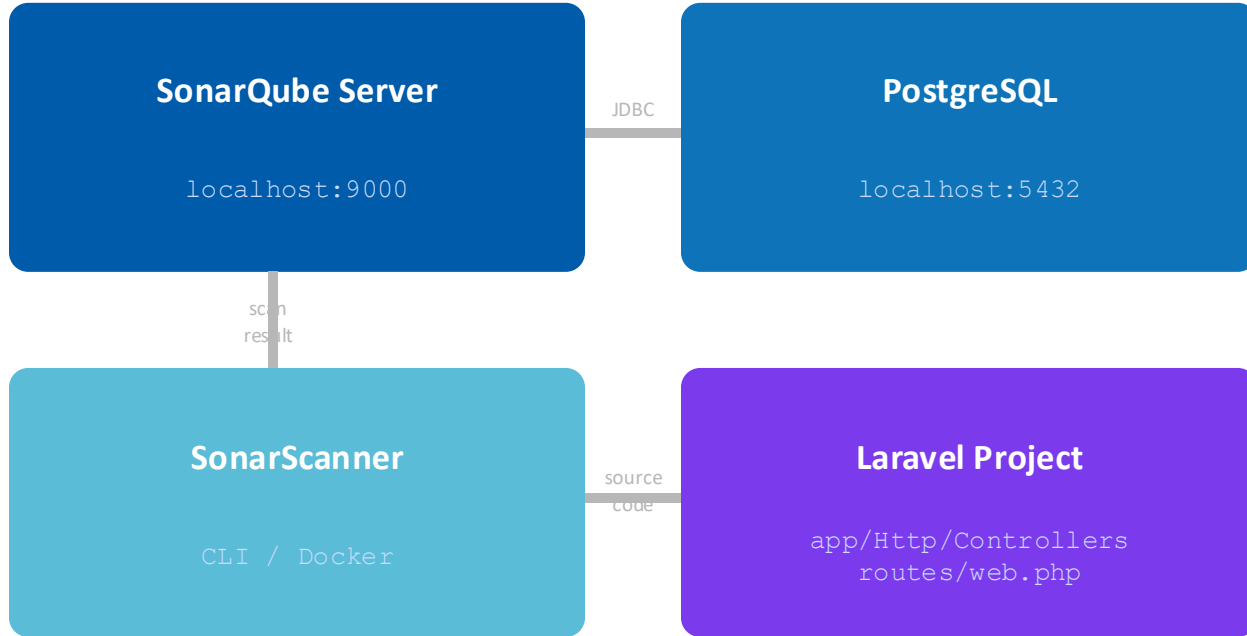
|                |         |
|----------------|---------|
| PHP / Composer | PHP 8.x |
|----------------|---------|

## SECTION 1

# Instalasi SonarQube

Setup environment menggunakan Docker Compose

# Arsitektur Lingkungan Praktikum



Semua service berjalan dalam Docker Network yang sama • Data tersimpan dalam named volumes

# Step 1 — Buat Folder Project & docker-compose.yml

## 1 Buat folder kerja

Terminal

```
mkdir sonarqube-lab && cd sonarqube-lab
```

## 2 Buat file docker-compose.yml

docker-compose.yml

```
version: "3"
services:
  sonarqube:
    image: sonarqube:community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_logs:/opt/sonarqube/logs
    depends_on:
```

## Step 2 — Jalankan SonarQube & Verifikasi

Terminal — Jalankan

```
docker compose up -d  
docker compose ps    # pastikan sonarqube & db "running"
```



### Checklist Verifikasi

✓ Buka browser → <http://localhost:9000> (default login: admin/admin)

✓ Ganti password default saat diminta

✓ Pastikan status kedua container "Up" di docker compose ps



Jika SonarQube gagal start: jalankan `sudo sysctl -w vm.max_map_count=262144`

## Step 3 — Setup Project di SonarQube Web UI

### 1 Create Project

Klik "Create project" → Pilih "Local project"  
Project key: laravel-sast-lab  
Display name: Laravel SAST Lab

### 3 Pilih Language

Pilih "Other (JS, TS, Go, Python, PHP, ...)" → "PHP"  
Salin konfigurasi sonar-project.properties yang ditampilkan

### 2 Generate Token

Buat token baru: "Provide a token"  
Simpan token — akan digunakan di sonar-project.properties

### 4 Pilih Scanner

Pilih "Use the SonarScanner" → salin perintah scan  
Simpan project-key dan token untuk langkah selanjutnya

## SECTION 2

# Konfigurasi Project Laravel & Scan

Membuat sample project dengan kerentanan yang disengaja

# Sample Project — Vulnerable Laravel Application

Project ini mengandung 6 kerentanan keamanan yang disengaja untuk tujuan pembelajaran:

UserController.php

CRITICAL

**SQL Injection via raw query**

AuthController.php

MAJOR

**Sensitive Data in Log**

UserController.php

MAJOR

**Mass Assignment Vulnerability**

routes/web.php

CRITICAL

**CSRF Protection Disabled**

AuthController.php

CRITICAL

**Hardcoded Credentials**

.env

MAJOR

**Debug Mode in Production (APP\_DEBUG=true)**

# File 1 — UserController.php (SQL Injection + Mass Assignment)

app/Http/Controllers/UserController.php

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use DB;

class UserController extends Controller
{
    // ⚠ VULNERABILITY: SQL Injection — input langsung di-concat ke query
    public function getUserById(Request $request)
    {
        $id = $request->input('id');
        $user = DB::select("SELECT * FROM users WHERE id = $id"); // BAHAYA!
        return response()->json($user);
    }

    // ⚠ VULNERABILITY: Mass Assignment — semua field diterima tanpa filter
    public function updateProfile(Request $request)
    {
        $user = auth()->user();
        $user->update($request->all()); // is_admin bisa diset oleh user!
        return response()->json(['status' => 'updated']);
    }
}
```



SQL Injection baris 13: \$id langsung dimasukkan ke query tanpa sanitasi → attacker bisa injeksi kode SQL sembarang

## File 2 — AuthController.php (Hardcoded Credentials + Sensitive Log)

app/Http/Controllers/AuthController.php

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;

class AuthController extends Controller
{
    // ⚠ VULNERABILITY: Hardcoded admin credentials
    private $adminEmail    = "admin@company.com";
    private $adminPassword = "Admin@123!";    // HARDCODED SECRET!

    public function login(Request $request)
    {
        $email    = $request->input('email');
        $password = $request->input('password');

        // ⚠ VULNERABILITY: Password plaintext dicatat di log
        Log::info("Login attempt: email=$email password=$password"); // SANGAT BERBAHAYA!

        if ($email === $this->adminEmail && $password === $this->adminPassword) {
            return response()->json(['token' => 'hardcoded-token-123']);
        }
        return response()->json(['error' => 'Invalid credentials'], 401);
    }
}
```



Baris 9-10: Credential hardcoded di source code → siapapun yang akses repository bisa melihatnya



Baris 18: Password ditulis ke log — log file sering tidak terenkripsi dan diakses banyak orang

## File 3 — routes/web.php & .env (CSRF Disabled + Debug Mode)

routes/web.php

```
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\UserController;
use App\Http\Controllers\AuthController;

// ⚠ VULNERABILITY: CSRF protection explicitly disabled for critical routes
Route::withoutMiddleware([\Illuminate\Foundation\Http\Middleware\VerifyCsrfToken::class])->group(function () {
    Route::post('/user/update', [UserController::class, 'updateProfile']);
    Route::post('/auth/login', [AuthController::class, 'login']);
});

Route::get('/user', [UserController::class, 'getUserById']);
```

.env

```
APP_NAME=Laravel
APP_ENV=production
APP_KEY=base64:xxx...
APP_DEBUG=true          # ⚠ VULNERABILITY: Debug mode aktif di production!
APP_URL=http://example.com

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
```

# Konfigurasi SonarQube — sonar-project.properties

sonar-project.properties (letakkan di root project Laravel)

```
# — Project Info —————
sonar.projectKey=laravel-sast-lab
sonar.projectName=Laravel SAST Lab
sonar.projectVersion=1.0

# — Server —————
sonar.host.url=http://localhost:9000
sonar.token=<TOKEN_DARI_WEB_UI> # Ganti dengan token Anda!

# — Source —————
sonar.sources=app, routes, config, database
sonar.tests=tests
sonar.language=php

# — Exclusions —————
sonar.exclusions=vendor/**, storage/**, public/vendor/**
sonar.test.inclusions=tests/**

# — PHP Version —————
sonar.php.version=8.1

# — Encoding —————
sonar.sourceEncoding=UTF-8
```

 **Jangan commit file sonar-project.properties yang mengandung token ke repository publik!**

# Step 4 — Install & Jalankan SonarScanner

## Opsi A — Menggunakan Docker (Direkomendasikan)

Terminal (dari root project Laravel)

```
docker run --rm \
  --network host \
  -v "$PWD:/usr/src" \
  sonarsource/sonar-scanner-cli \
  -Dsonar.projectBaseDir=/usr/src
```

## Opsi B — Install SonarScanner Secara Langsung

Terminal

```
# Download dan extract SonarScanner
wget https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-5.0.1.3006-linux.zip
unzip sonar-scanner-cli-*.zip
export PATH=$PATH:$PWD/sonar-scanner-*/bin

# Jalankan dari root project Laravel
cd ~/laravel-app
sonar-scanner
```



Scan berhasil jika muncul: "EXECUTION SUCCESS" di terminal dan hasil muncul di <http://localhost:9000>

## SECTION 3

# Membaca Laporan & Findings

Interpretasi hasil scan SonarQube pada project Laravel

# Overview Dashboard SonarQube



**Quality Gate: FAILED**

3

Bugs

4

Vulnerabilities

2

Security Hotspots

8

Code Smells

0.0%

Coverage

0.0%

Duplications

## Temuan Kritis yang Harus Segera Diperbaiki:



SQL Injection di UserController.php:13 — CRITICAL



CSRF protection disabled di routes/web.php:7 — CRITICAL



Hardcoded credentials di AuthController.php:9 — CRITICAL

# Kategori Findings di SonarQube



## Bugs

Kode yang pasti salah atau crash saat runtime.

Contoh pada project: null pointer di hasil query yang mungkin kosong.



## Vulnerabilities

Celah keamanan yang dapat dieksploitasi attacker.

Contoh: SQL Injection, Hardcoded Credentials, CSRF disabled.



## Security Hotspots

Kode sensitif yang perlu ditinjau manusia.

Contoh: penggunaan `Log::info()` dengan data sensitif.



## Code Smells

Masalah maintainability — kode bekerja tapi sulit dipelihara.

Contoh: method terlalu panjang, nama variabel tidak deskriptif.

# Severity Levels — Prioritas Remediasi

**BLOCKER****P0**

Harus diperbaiki sebelum release. Akan menyebabkan crash atau data breach parah.

**CRITICAL****P1**

Prioritas sangat tinggi. SQL Injection, Hardcoded Credentials, CSRF disabled masuk di sini.

**MAJOR****P2**

Perlu segera ditangani. Mass Assignment, Debug mode di production, sensitive log.

**MINOR****P3**

Pengaruh kecil ke security. Perbaiki di sprint berikutnya.

**INFO****P4**

Informasi / saran style. Tidak berdampak langsung ke keamanan.

# Temuan & Remediasi — SQL Injection (Laravel)

**CRITICAL**

UserController.php, baris 13 — OWASP A03:2021 Injection

## BEFORE — Kode Rentan

```
// BEFORE — String concatenation langsung!  
$id    = $request->input('id');  
$user = DB::select("SELECT * FROM users WHERE id =  
$id"); // SQL INJECTION!
```

## AFTER — Kode Aman

```
// AFTER — Gunakan Query Builder / Eloquent!  
$id    = $request->input('id');  
  
// Opsi 1: Query Builder (parameterized)  
$user = DB::select("SELECT * FROM users WHERE id = ?",  
[$id]);  
  
// Opsi 2: Eloquent ORM (direkomendasikan)
```

## Mengapa ini berhasil?

- Query Builder dan Eloquent menggunakan PDO Prepared Statements secara otomatis.
- Parameter ([ \$id ]) dikirim terpisah dari SQL — database TIDAK bisa menginterpretasikannya sebagai perintah SQL.
- Eloquent ORM menambah lapisan abstraksi yang mencegah raw SQL langsung.

# Temuan & Remediasi — Mass Assignment (Laravel)

## MAJOR

### UserController.php — Unprotected Mass Assignment

#### BEFORE — Kode Rentan

```
// BEFORE — $request->all() tanpa filter
$user = auth()->user();
$user->update($request->all()); // Attacker bisa set
is_admin=1!
```

#### AFTER — Kode Aman

```
// AFTER — Whitelist fields yang boleh diupdate
$user = auth()->user();

// Opsi 1: Gunakan $fillable di Model + only()
$user->update($request->only(['name', 'email',
'phone']));

// Opsi 2: Gunakan Form Request Validation (best
practice)
```

#### Model — User.php (Tambahkan \$fillable)

```
// app/Models/User.php
class User extends Model
{
    // ✅ Definisikan $fillable untuk mencegah mass assignment
    protected $fillable = ['name', 'email', 'phone'];

    // Atau gunakan $guarded untuk blacklist
    protected $guarded = ['is_admin', 'role'];
}
```

# Temuan & Remediasi — CSRF Disabled & Debug Mode

## 1 CSRF Protection Disabled

routes/web.php — BEFORE

```
// BEFORE — Middleware CSRF di-disable
Route::withoutMiddleware([VerifyCsrfToken::class]) -
>group(function () {
    Route::post('/user/update', [UserController::class,
        'updateProfile']);
});
```

routes/web.php — AFTER

```
// AFTER — Hapus withoutMiddleware, biarkan CSRF aktif
// Pastikan form HTML mengandung @csrf
Route::post('/user/update', [UserController::class,
    'updateProfile']);

// Di view Blade:
<form method="POST" action="/user/update">
    @csrf {{-- Token CSRF otomatis disisipkan --}}
</form>
```

## 2 Debug Mode di Production

.env — BEFORE

```
# BEFORE .env
APP_DEBUG=true # Stack trace terekspos ke user!
APP_ENV=production
```

.env — AFTER

```
# AFTER .env
APP_DEBUG=false # Sembunyikan error detail dari user
APP_ENV=production
LOG_LEVEL=error # Log hanya error, bukan debug info
```



Jika `APP_DEBUG=true` di production, stack trace Laravel (termasuk env vars!) bisa terekspos kepada user saat terjadi error.

# Lembar Kerja Kelompok — Dokumentasi Findings

Isikan temuan SonarQube project Laravel kelompok Anda pada tabel di bawah ini:

| No | File | Baris | Kategori | Severity | Deskripsi Temuan | Solusi / Remediasi |
|----|------|-------|----------|----------|------------------|--------------------|
| 1  |      |       |          |          |                  |                    |
| 2  |      |       |          |          |                  |                    |
| 3  |      |       |          |          |                  |                    |
| 4  |      |       |          |          |                  |                    |
| 5  |      |       |          |          |                  |                    |
| 6  |      |       |          |          |                  |                    |

# Tugas — Laporan Praktikum

Buat laporan praktikum dalam format PDF dengan struktur berikut:

## BAB I — Pendahuluan

Latar belakang, tujuan praktikum, manfaat yang diharapkan.

## BAB II — Landasan Teori

Konsep SAST, pengenalan SonarQube, Quality Gate, OWASP Top 10 relevan (A01 Access Control, A02 Cryptographic Failures, A03 Injection).

## BAB III — Langkah Kerja

Dokumentasi instalasi SonarQube, konfigurasi Laravel project, proses scan (screenshot tiap langkah).

## BAB IV — Hasil & Analisis

Screenshot dashboard, tabel semua findings, analisis root cause tiap kerentanan, severity dan dampaknya.

## BAB V — Remediasi

Kode sebelum dan sesudah perbaikan untuk setiap finding, penjelasan teknis mengapa solusi efektif.

## BAB VI — Kesimpulan

Rangkuman temuan, lesson learned, rekomendasi best practice untuk development Laravel yang aman.

# Rubrik Penilaian Praktikum

| N<br>o                                                                             | Aspek Penilaian         | Kriteria Penilaian                                            | Bobot |
|------------------------------------------------------------------------------------|-------------------------|---------------------------------------------------------------|-------|
| 1                                                                                  | Instalasi & Konfigurasi | SonarQube berjalan, project Laravel berhasil di-scan          | 20%   |
| 2                                                                                  | Identifikasi Temuan     | Semua findings terdokumentasi lengkap (file, baris, severity) | 25%   |
| 3                                                                                  | Analisis & Root Cause   | Penjelasan teknis mengapa kode rentan dan dampak potensialnya | 25%   |
| 4                                                                                  | Remediasi Kode          | Perbaikan kode tepat, sesuai best practice Laravel/PHP        | 20%   |
| 5                                                                                  | Laporan & Presentasi    | Laporan lengkap, struktur baik, presentasi jelas              | 10%   |
|  | TOTAL                   |                                                               | 100%  |

# Referensi & Penutup

---

-  SonarQube Documentation — [docs.sonarqube.org](https://docs.sonarqube.org)
-  Laravel Security Best Practices — [laravel.com/docs](https://laravel.com/docs)
-  OWASP Top 10 2021 — [owasp.org/Top10](https://owasp.org/Top10)
-  PHP Security Guide — [phptherightway.com/#security](https://phptherightway.com/#security)
-  Composer Security Advisories — [packagist.org/security-advisories](https://packagist.org/security-advisories)



## Selamat Praktikum!

Gunakan ilmu ini untuk  
membangun aplikasi  
PHP/Laravel yang lebih aman

DevSecOps — STT Nurul Fikri